

DATA ENGINEERING

Binary Search Trees

Algorithm Design & Pythonic Implementation

Module: 05

Series: CodeHive Pro

01. Defining the BST

A **Binary Search Tree (BST)** is an advanced, ordered variant of the binary tree. It is optimized for speed, allowing for near-instant retrieval, insertion, and deletion of data without the memory-intensive "shifting" required by arrays.

The BST Invariant

For any node X in a BST, three properties must hold true:

1. **Left Constraint:** Every node in X's left subtree has a value **less than** X.
2. **Right Constraint:** Every node in X's right subtree has a value **greater than** X.
3. **Recursive Logic:** Both the left and right subtrees must themselves be valid Binary Search Trees.

Assumption: To maintain simplicity in our primary logic, we assume all values in a BST are unique.

02. Verification & Traversal

The most elegant way to verify if a tree is a valid BST is to perform an **In-order Traversal**. Because of the Left < Root < Right rule, an in-order traversal will naturally visit the nodes in strictly increasing order.

```
class TreeNode:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

def inOrderTraversal(node):
    # Recursive verification
    if node is None:
        return
    inOrderTraversal(node.left)
    print(node.data, end=" ")
    inOrderTraversal(node.right)
```

If the output is [3, 7, 8, 13, 14, 15, 18, 19], the tree is functionally sound and ordered.

03. Searching Algorithm

Searching a BST follows a "divide and conquer" strategy similar to binary search on an array. With every comparison, you eliminate half of the remaining tree.

Operational Steps:

- Compare the **target** with the **current node**.
- If target is **smaller**, discard the right side and move Left.
- If target is **larger**, discard the left side and move Right.

```
def search(node, target):  
    if node is None or node.data == target:  
        return node  
  
    if target < node.data:  
        return search(node.left, target)  
    else:  
        return search(node.right, target)
```

The time complexity of this operation is **$O(h)$** , where h is the height of the tree.

04. Insertion Strategy

In a BST, the structural integrity is maintained by placing new data at the first available leaf position that satisfies the ordering rule.

```
def insert(node, data):  
    # Base Case: Found the insertion point  
    if node is None:  
        return TreeNode(data)  
  
    # Navigate the tree structure  
    if data < node.data:  
        node.left = insert(node.left, data)  
    elif data > node.data:  
        node.right = insert(node.right, data)  
  
    return node
```

Fact: Inserting into a BST always results in the new value becoming a **leaf node** initially. It never "pushes" existing nodes down.

05. Finding Subtree Minimums

Before handling complex deletions, we need a utility to find the **In-order Successor**. The successor is the smallest value in the right subtree.

The Logic

To find the lowest value in any BST subtree, you simply start at the local root and travel **Left** as far as possible until you reach a node with no left child.

```
def minValueNode(node):  
    current = node  
    # Keep diving left ...  
    while current.left is not None:  
        current = current.left  
    return current
```

This is a crucial helper function for Case 3 of our Deletion algorithm.

06. Deletion: The Simple Cases

Deleting from a BST is more complex than insertion because we must ensure the "ordering rule" isn't broken when a node vanishes.

Case 1: Leaf Node Deletion

If the node has no children, we simply return None to its parent, effectively snipping the link.

Case 2: Single Child Deletion

If the node has one child (either left or right), we bypass the deleted node and link the parent directly to the child—similar to deleting in a linked list.

```
# Logic snippet inside delete function:
if not node.left:
    return node.right
elif not node.right:
    return node.left
```

07. Deletion: The Complex Case

When a node has **both** a left and right child, we cannot simply "bypass" it. We must find a replacement that keeps the order intact.

The Replacement Trick

1. Find the node's **In-order Successor** (the smallest node in the right branch).
2. Copy that successor's value into the target node.
3. Delete the original successor node (which is guaranteed to be a leaf or have only one child).

```
# Case 3 Implementation
successor = minValueNode(node.right)
node.data = successor.data
node.right = delete(node.right, successor.data)
```

08. BST vs. Other Data Structures

Why choose a BST? It occupies the "sweet spot" of performance, combining the search speed of a sorted array with the insertion speed of a linked list.

Data Structure	Search Speed	Memory Shifting?
Sorted Array	$O(\log n)$	Yes (Slow Insert)
Linked List	$O(n)$	No (Fast Insert)
Binary Search Tree	$O(\log n)^*$	No

*Assuming the tree is balanced.

09. Complexity & Balance

The efficiency of a BST is tied directly to its **Height (h)**. All operations (Search, Insert, Delete) are **$O(h)$** .

The Danger of Unbalance

If you insert values in order (1, 2, 3, 4, 5), the tree becomes a single long path. In this "degenerate" case, **$h = n$** and the BST becomes a glorified Linked List with **$O(n)$** performance.

The Solution: AVL Trees

On the next level of training, you will encounter **Self-Balancing Trees (AVL)**. These trees perform "rotations" during every insertion to ensure the height always stays around **$\log n$** .

The CodeHive Standard

© 2026 CodeHive Tutorials. Excellence in Algorithmic Education.